SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



13. Generische Programmierung in C++ mit Schablonen & Konzepten

13.1 C++ Schablonen
(templates)

Einleitung Templates

- generische Programmierung in C++ in From von Vorlagen/Schablonen (*templates*) für
  - Funktionen
  - Klassen
  - Variablen (seit C++14)
- Compiler erstellt eigentlichen Quelltext selbst, z.B. für verschiedene Typen
  - Templates existieren nur zur Compile-Zeit
  - erstellte Varianten stehen in keiner Beziehung zueinander

Vorteile

- vermindert Code-Redundanz und erhöht Wartbarkeit
- reduziert Rechenaufwand zur Laufzeit
- erlaubt Template-Metaprogramming[a]

Nachteile

- schwerer lesbar
- komplizierte Fehlermeldungen
- langsamere Übersetzung
- größere Binaries (Code wird mehrfach instantiiert)

[a] Programme die zur Compile-Zeit ausgeführt werden und Code erzeugen

guter Überblick F. Busato: Modern C++ Programming Kapitel: Templates

Template Syntax für Funktionen

- Schlüsselwort `template` mit Parametern in spitzen Klammern `<>`
- Art der Template-Parameter
  - Konstanten eingeleitet durch Datentyp, z.B. `int`
  - Typ-Name (häufigste Art)
    - eingeleitete durch `typename` oder `class`: beliebiger Datentyp
    - *Konzept*-Name: Typ mit bestimmten Voraussetzungen (seit C++20)
  - Template-Template Parameter:
    - Parameter ist selbst ein Template, z.B. generischer Container

```cpp
template<int Length = 3>
void fconst(std::array<int, Length> a) {}

template<typename T> // or template<class T>
void ftype(T value) {}

template<std::integral T>
void fconcept(T value) {}

template<template<typename> typename Container>
void ftemplate(const Container<int> &c) {}
```

```cpp
int main() {
  fconst<4>({1, 2, 3, 4});
  fconst<2>({1, 2});

  ftype(1.0);
  ftype('a');
  ftype(new int[10]{1, 2, 3});

  fconcept(1);
  // fconcept("a"); // XXX: not integer type

  ftemplate<std::vector>({1, 2, 3});
}
```

Template parameters - cppreference.com

Beispiel: Funktionstemplate

```
template <typename T>
void print(T const& x) { P("<>{}, ", x); }

// non-template regular function
void print(const char *x) { P("{}", x); }

int main() {
  print<int>(1); //< explicit template & type
  print<>(1);    //< template & automatic type
  print(1);      //< automatic template
                                      PL();
  print(2.2);
  print<int>(3.3); // explicit type
  print('c');
                                      PL();
  print<>("template"); // templated function
  print("normal"); // regular function
}
```

```
<>1, <>1, <>1,
<>2.2, <>3, <>c,
<>template, normal
```

generierte Funktionen

```
void print(int const& x) {
  P("<>{}, ", x);
}

void print(double const& x) {
  P("<>{}, ", x);
}

void print(char const& x) {
  P("<>{}, ", x);
}

void print(const char* const& x) {
  P("<>{}, ", x);
}
```

- nicht-template Funktion wird bei Aufruf bevorzugt (falls vorhanden)

Beispiel: Klassentemplate

```
template <typename T>
class vector {
  T* m_data;
  size_t m_size;
  size_t m_capacity;

public:
  explicit vector(size_t capacity = 0);

  vector(const vector &);

  vector& operator = (const vector&);

  ~vector() { delete m_data; }

  T& operator [] (size_t i);

  friend void swap(vector &a, vector &b);

  // ...
};
```

```
template <typename T>
vector<T>::vector(size_t capacity)
  : m_data(new T[capacity]), m_size(0),
    m_capacity(capacity) {
}

template <typename T>
vector<T>::vector(const vector &o) {
  // ...
}

template <typename T>
T& vector<T>::operator [] (size_t i) {
  return m_data[i];
}

// ...
```

Explizite Spezialisierung & Löschen von Spezialisierungen

- explizite Spezialisierungen (`template<>`) erstellen oder löschen (`= delete`)

```
template <typename T>
void print(T const& x) { PL("generic {}", x); }

template<>
void print(int const& x) { PL("int {}", x); }

template<>
void print(const char* const& x) = delete;

int main() {
  print(1); // special int version
  print(2.2); // generic
  print('c'); // generic
  // print("hello"); // XXX: deleted function
}
```

```
int 1
generic 2.2
generic c
```

- nur explizite Spezialisierungen

```
template <typename T>
void print(T const& x) = delete;

template<>
void print(int const& x) { PL("int {}", x); }

template<>
void print(char const& x) { PL("char {}", x); }

int main() {
  print(1); // special int version
  print('c'); // special char version
  // print(2.2); // XXX: deleted function
  // print("hello"); // XXX: deleted function
}
```

```
int 1
char c
```

- ermöglicht eine Art Polymorphismus ohne Vererbung (Basis-Schablone als Interface)

Bessere Typsicherheit als Makros

- z.B. sicherstellen, dass alle Parameter gleichen Typ haben

- bei Makros keine Typ-Überprüfung, erlaubt Vergleich inkompatibler Typen mit fragwürdigem Ergebnis

```
#define MAX(X, Y) ((X) > (Y)) ? (X) : (Y)

int main() {
  SHOW(MAX(1, 2));
  SHOW(MAX('a', 'b'));
  SHOW(MAX('a', 1)); // incompatible types
  SHOW(MAX(-1, 2u)); // incompatible types
}
```

```
MAX(1, 2) = 2
MAX('a', 'b') = 'b'
MAX('a', 1) = 97
MAX(-1, 2u) = 4294967295
```

- Schablone kann Gleichheit aller Typen »`T`« erzwingen

```
template<typename T>
T max(T a, T b) {
  return a > b ? a : b;
}

int main() {
  SHOW(max(1, 2));
  SHOW(max('a', 'b'));
  // SHOW(max('a', 1)); // XXX: max(char, int)
  // SHOW(max(-1, 2u)); // XXX: max(int, unsigned)
}
```

```
max(1, 2) = 2
max('a', 'b') = 'b'
```

Übersetzung erst bei Instantiierung

- Template alleine erzeugt keinen Code → kann Fehler verbergen[1]
  - explizite Instantiierung durch Deklaration mit Typ-Argument `template int broken<int>(int)`
  - implizite Instantiierung durch Aufruf `broken(1)`

```
template <typename T>
T broken(T x) { // NOTE: template itself is not compiled (yet)
  x + x = x; // invalid code may not be detected
}
// ----- up to this point: no compiler error ----------

template int broken<int>(int); //< explicit specialization --> error is found

int main() {
  broken(1); //< implicit specialization --> error is found
}
```

```
In instantiation of ‘T broken(T) [with T = int]’:
required from
  broken(1)   and   template int broken<int>(int);
  ~~~~~~^~~                                   ^
error: lvalue required as left operand of assignment
  x + x = x;
  ~~^~~
```

[1] Die Addition könnte (theoretisch) als Ergebnis einen lvalue produzieren

Typinferenz mit `decltype` & `auto`

- `decltype` ermöglicht Ableitung des Typs eines Ausdrucks
- `auto` erlaubt Typinferenz zur Compile-Zeit

```
int i = 1;

decltype(i * 1.0) d; //< type of int*double
d = i * 2.2;

auto a = i;

SHOW(i, d, a);
```

```
i = 1; d = 2.2; a = 1
```

- z.B. Typ des Ergebnisses von `T1{} + T2{}` ableiten durch Aufruf der beiden Default-Konstruktoren (`T1{}` und `T2{}`)

```
template <typename T1, typename T2>
decltype(T1{} + T2{}) add(T1 a, T2 b) {
  return a + b;
}

// since C++14
template<typename T1, typename T2>
auto add2(T1 a, T2 b) {
  return a + b;
}

// or even without template keyword
auto add3(auto a, auto b) {
  return a + b; // -> macro-like type (un)safety
}

int main() {
  SHOW(add('a', 13l));
  SHOW(add2('a', 13l));
  SHOW(add3('a', 13l));
}
```

```
add('a', 13l) = 110
add2('a', 13l) = 110
add3('a', 13l) = 110
```

Werte als Template-Argumente

- primitive Datentypen können auch als Wert übergeben werden
- erlaubt Berechnungen zur Compile-Zeit → ermöglicht “Template Metaprogrammierung (TMP)”
  - Turing-vollständige Programmiersprache (zufällig entdeckt)
  - in C++17 und C++20 durch modernere Konzepte großteils abgelöst

```cpp
template <int a, int b>
int add() { return a + b; }

int main() {
  SHOW(add<3,4>());
}
```

```
add<3,4>() = 7
```

```cpp
template<typename T, size_t len>
size_t array_len(std::array<T, len> a) {
  return len;
}

int main() {
  auto a = std::to_array<int>({1, 2, 3});
  SHOW(array_len(a));
}
```

```
array_len(a) = 3
```

Default Werte für Template-Parameter

- Default Werte für Template-Parameter (wie bei Funktionsparameter)

```
template <typename T = int>
std::vector<T> make_vector() {
  return std::vector<T> {};
}

int main() {
  auto v = make_vector();
  auto v2 = make_vector<double>();
  SHOW(v, v2);
}
```

```
v = []; v2 = []
```

- abgeleitete Werte von vorherigen Parametern (seit C++11)

```
template <typename T1 = int, typename T2 = T1>
std::pair<T1, T2> make_pair() {
  return std::pair<T1, T2>{};
}

int main() {
  auto p1 = make_pair();
  auto p2 = make_pair<double>();
  auto p3 = make_pair<double, int>();
  SHOW(p1, p2, p3);
}
```

```
p1 = (0, 0); p2 = (0, 0); p3 = (0, 0)
```

Template-Parameter mit `auto`

- vor C++17 Angabe von Parameter Typ als eigener Template Parameter

```
template <typename T1,   T1 a,
          typename T2,   T2 b>
void test() {
  PL("test<{}, {}>()", a, b);
}

int main() {
  test<int, 1,
       int, 2>();

  test<unsigned, 3u,
       long,     4l>();

  //! XXX: not a primitive data type
  // test_pre17<std::string, "a",
  //            std::string, "b">();
}
```

```
test<1, 2>()
test<3, 4>()
```

- seit C++17 automatische Ableitung des primitiven Datentyps mit `auto`

```
template<auto a, auto b>
void test() {
  PL("test<{}, {}>()", a, b);
}

int main() {
  test<1, 2>();
  test<3u, 4l>();

  //! XXX: not a primitive data type
  // test<"a", "b">();
}
```

```
test<1, 2>()
test<3, 4>()
```

Typ-Schablonen mit `using`

- im Unterschied zu `typedef` kann `using` auch mit Templates kombiniert werden

```
template <typename T>
using container_type = std::vector<T>;

int main() {
  auto c1 = container_type<double> { 1, 2, 3 };
  //! T inferred -> int
  auto c2 = container_type { 1, 2, 3 };
  SHOW(c1, c2);
}
```

```
c1 = [1, 2, 3]; c2 = [1, 2, 3]
```

```
template <typename IN, typename OUT>
using function_type = OUT (*)(IN);

int char_to_int(char c) { return c; }
char int_to_char(int i) { return i; }

int main() {
  function_type<char,int> f1 = char_to_int;
  function_type<int,char> f2 = int_to_char;
  SHOW(f1('a'), f2(100));
}
```

```
f1('a') = 97; f2(100) = 'd'
```

weitere Beispiele: Funktionstemplates

- z.B. generische Swap-Funktion

```
template<typename T>
void swap(T& a, T& b) {
  T temp = a;
  a = b;
  b = temp;
}

int main() {
  char a = 'a', b = 'b';
  swap(a, b); SHOW(a, b);

  int i = 1, j = 2;
  swap(i, j); SHOW(i, j);

  auto s1 = "one", s2 = "two";
  swap(s1, s2); SHOW(s1, s2);
}
```

```
a = 'b'; b = 'a'
i = 2; j = 1
s1 = "two"; s2 = "one"
```

- eigene Funktion pro Array-Typ und Array-Länge

```
template <typename T, size_t S>
void print(const T (&a)[S]) {
  std::println("array len = {}: ", S);
  for (const auto& e: a) {
    std::print("{}, ", e);
  }
  std::println();
}

int main() {
  int a[] = {1, 2, 3};
  print(a);
  const char* s[] = {"hello", "world"};
  print(s);
}
```

```
array len = 3:
1, 2, 3,
array len = 2:
hello, world,
```

Rekursive Funktionstemplates mit *Parameter Packs*

- durch Parameter Packs können auch Template-Funktionen mit variabler Parameteranzahl definiert werden

```
void print() {
  std::println();
}

template <typename T, typename... Types>
void print(T first, Types... rest) {
  std::print("{}, ", first);
  print(rest...);
}


int main() {
  print(1);
  print(1, 'a');
  print(1, 'a', "ABC");
}
```

```
1,
1, a,
1, a, ABC,
```

erlauben variable Anzahl von Parametern mit unterschiedlichen Typen

- folding expressions für binäre Operatoren seit C++17

```
template <typename... Targs>
auto add_all(Targs... args) {
  return (0 + ... + args);
}

int main() {
  SHOW(add_all(1, 2, 3u, 4l, 5ll));
}
```

```
add_all(1, 2, 3u, 4l, 5ll) = 15
```

13.2 Klassen-Schablonen
(class templates)

Klassen-Schablonen

- ähnlich zu Funktionsschablonen (*function templates*)
- erlauben zusätzlich Teil-Spezialisierung

```
// classes
template <typename T, typename X> class C {};
template <typename T>             class C<T, int> {}; // partial specialization
template <>                       class C<int, int> {}; // full specialization

// functions
   template <typename T, typename X> void f(T t, X x) { PL("generic"); }
// template <typename T>             void f<T,int>(T t, int x) {} // XXX: not allowed
   template <>                       void f<int,int>(int t, int x) { PL("special"); }

int main() {
  C<char, char> c1;
  C<char, int> c2;
  C<int, int> c3;
  f('a', 'b');
  f('a', 1);
  f(1, 1);
}
```

```
generic
generic
special
```

Klassen-Template Argumente

- Eigener Klasse-Template-Parameter muss nicht wiederholt werden
- zusätzliche Schablonen für Methoden möglich

```
template <typename T>
class C {
  C();
  C(const C &c);
  // equivalent to
  C(const C<T> &c);

  C<T>& operator = (const C<T>&);
  // equivalent to
  C& operator = (const C&);

  template<typename Other>
  C(const Other &o); // converting constructor(s)
};
```

- Implementierung außerhalb der Deklaration erfordert wieder `template <typename T>`

```
template<typename T>
C<T>::C() { /* ... */ }

template<typename T>
C<T>::C(onst C &c) { /* ... */ }

template<typename T>
template<typename Other>
C<T>::C(const Other &o) { /* ... */ }
```

aber: Schablone erzeugt keinen Code → Klassen-Template inkl. Implementierung meist im Header notwendig

Constructor Template Automatic Deduction (CTAD)

- seit C++17 kann Compiler Template Parameter oft selbst ableiten
- für Grenzfälle können Ableitungsrichtlinien (*deduction guides*) erstellt werden

```
template <typename T>
struct Point {
  T x {};
  T y {};
  Point(T x = {}, T y = {}) : x(x), y(y) {}
};

// deduction guide
template <typename T>
Point(double, T) -> Point<double>;

int main() {
  Point pi(1, 2);
  Point pd(1.0, 2.0);

  Point pg(1.0, 2); // needs deduction guide
}
```

13.3 Typ-Eigenschaften
(type traits)

Type-Traits

- Klassen können nicht nur statische und dynamische *Daten*-Komponenten
- sondern auch Typ-Komponenten besitzen
- innere Typen nach Außen weitergeben (re-exportieren)
- erleichtert Schreiben von generischen Funktionen

```
template <typename T>
class vector {
  public:
    using value_type = T;
  // ...
};

template <typename T>
T::value_type get_first(T container) {
  typename T::value_type first_element = container[1];
  return first_element;
}

int main() {
  std::vector<int> v { 1, 2, 3 };
  SHOW(get_first(v));
}
```

```
get_first(v) = 2
```

Eigene Type-Traits

```cpp
template <typename T> struct is_integer {
  static constexpr bool value = false;
};
template <> struct is_integer<int> {
  static constexpr bool value = true;
};

struct Complex {};

template <> struct is_integer<Complex> {
  static constexpr bool value = true;
};

int main() {
  SHOW(is_integer<int>::value);
  SHOW(is_integer<double>::value);
  SHOW(is_integer<Complex>::value);
}
```

```
is_integer<int>::value = true
is_integer<double>::value = false
is_integer<Complex>::value = true
```

- Definition der Type-Traits in Standardbibliothek über entsprechende Meta-Typen

```cpp
template <typename T1, typename T2>
struct is_same {
  static constexpr bool value = false;
};

template <typename T>
struct is_same<T, T> {
  static constexpr bool value = true;
};

int main() {
  SHOW(is_same<int, int>::value);
  SHOW(is_same<int, double>::value);
}
```

```
is_same<int, int>::value = true
is_same<int, double>::value = false
```

Direkte Type-Traits Konventionen

- Verschiedene Typ-Arten beschreiben erforderliche Type-Traits
- z.B. für eigene C++ Container

```
template <typename T>
struct MyContainer {
  using value_type = T;
  using reference = &value_type;
  using const_reference = const reference;
  using iterator = ...;
  using const_iterator = ...;
  using difference_type = ...;
  using size_type = ...;
};
```

Statische “Code-Analyse” mit `static_assert()` – Introspection

- `assert` prüft zur Laufzeit beliebige Invarianten
- `static_assert` prüft bereits zur Übersetzungszeit
  - z.B. ob Typen kompatibel sind, oder die Größe passt

```
template <typename T>
T next_value(T n) {
  assert(n + 1 > n); // "value overflow"
  static_assert(std::numeric_limits<T>::is_exact, "inexact value");
  return n + 1;
}

int main() {
  next_value(1);
  next_value(std::numeric_limits<unsigned int>::max()); // XXX: runtime  error -> assertion failed
  next_value(1.1); // XXX: compile error -> inexact value
}
```

Type Traits: Typ-Eigenschaften abfragen

- in Header <type_traits> finden sich viele Prädikate die zur Übersetzungszeit verwendet werden können (`constexpr`), z.B. `is_integral`, `is_const`, `is_trivially_copyable`

```
SHOW(std::is_floating_point<int>::value);
SHOW(std::is_class_v<int>); // is_class<int>::value
SHOW(std::is_same_v<int, int>);
```

```
std::is_floating_point<int>::value = false
std::is_class_v<int> = false
std::is_same_v<int, int> = true
```

```
template <typename Dest, typename Src>
Dest saturating_cast(Src x) {
  static_assert(std::convertible_to<Src, Dest>);

  using l = std::numeric_limits<Dest>;
  if (x > l::max()) return l::max();
  if (x < l::min()) return l::min();

  return static_cast<Dest>(x);
}
```

```
int main() {
  auto n = static_cast<uint8_t>(1000);
  auto s = saturating_cast<uint8_t>(1000);
  SHOW(n, s);
  // saturating_cast<std::string>(1000); // XXX
}
```

```
n = 232; s = 255
```

```
In instantiation of 'Dest saturating_cast(Src)
  [with Dest = std::string<char>; Src = int]':
  required from here
   auto s = saturating_cast<std::string>(1000);
            ~~~~~~~~~~~~~~~~~~~~~~~~~~~~^~~~~~
error: static assertion failed
   static_assert(std::convertible_to<Src, Dest>);
                 ~~~~~^~~~~~~~~~~~~~~~~~~~~~~~
```

Type Traits: Typ-Eigenschaften ableiten

```cpp
template <typename T>
void unwrap(T &ptr) {
  using copy_t = std::remove_pointer<T>::type;
  copy_t cp = *ptr; // int const

  using unsigned_t = std::make_unsigned<copy_t>::type;
  unsigned_t u = cp; // unsigned const

  using mutable_t = std::remove_const<copy_t>::type;
  mutable_t m = cp; // unsigned
  m += 1;

  SHOW(cp, u, m);
}
int main() {
  int const i = 1;
  int const * const ptr = &i;
  unwrap(ptr);
}
```

```
cp = 1; u = 1; m = 2
```

Mit Hilfe von Type-Traits können abgeleitete Typen definiert werden, z.B. um den dahinter liegenden Basis-Typ zu finden

- `make_signed`, `make_unsigned`
- `remove_pointer`, `add_pointer`
- `remove_reference`, `add_reference`
- `remove_const`, `add_const`
- `common_type<R, S>`
- `common_reference<R, S>`
- ...

13.4 Konzepte
(concepts)

C++ Konzepte (*concepts*)

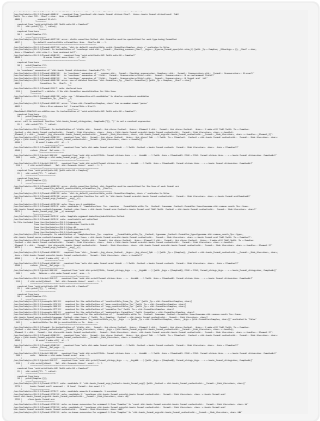
- Seit C++20 Erweiterung (und Vereinfachung) von Schablonen durch Konzepte `concept`
  - erlauben Anforderungen an Typ-Variablen `requires`
- Problem: Schablonen verwenden Duck Typing

- Anforderungen an generischen Parameter-Typ sind implizit über Verwendung
- oft schlecht verständliche Fehlermeldungen
- erlauben möglicherweise unsinnige oder nicht gewollte Verwendung

```
void print(auto value) {
  std::print("{}, ", value);
}

struct Complex {};

int main() {
  print(1);
  print('a');
  print("abc");
  // print(Complex {}); // XXX: compiler error --->
}
```



Constraints and concepts (since C++20) - cppreference.com

C++ Konzepte Einführung

- Spezifikation der Anforderungen an einen Typ `T`
  - durch Prädikate (`constexpr -> bool`)
  - durch “Übersetzbarkeit” (früher durch *Substitution Failure is not an Error (SFINAE)*)

```cpp
template<typename T>
concept arithmetic =
  std::is_arithmetic_v<T>; // returns true/false

template<arithmetic T>
T add_arithmetic(T a, T b) { return a + b; }

template<typename T> // NOTE: no type requirements
T add(T a, T b) { return a + b; }
```

```cpp
int main() {
  SHOW(add(1, 2));
  SHOW(add_arithmetic(1, 2));

  std::string a = "a", b = "b";
  SHOW(add(a, b));
  // SHOW(add_arithmetic(a, b)); // XXX
}
```

```
add(1, 2) = 3
add_arithmetic(1, 2) = 3
add(a, b) = "ab"
```

Anforderungen (*requires*)

- beliebige Anforderungen mit `requires` zusammenfassen und benennen

```
template <typename T>
concept arithmetic = requires (T a, T b) {
  { a + b } -> std::convertible_to<T>;
  { a - b } -> std::convertible_to<T>;
  { a * b } -> std::convertible_to<T>;
  { a / b } -> std::convertible_to<T>;
  { -a } -> std::convertible_to<T>;
};

template<arithmetic T>
T add(T a, T b) { return a + b; }

int main() {
  SHOW(add(1, 2));
  SHOW(add('1', '2'));
}
```

```
add(1, 2) = 3
add('1', '2') = 'c'
```

- z.B. arithmetische Operationen und Ergebnistyp voraussetzen

```
int main() {
  add("a", "b");
}
```

```
no matching function for call to
              'add(const char [2], const char [2])'
   add("a", "b");
   ~~~^~~~~~~~~~
there is 1 candidate: 'template<class T>  requires
                                 arithmetic<T> T add(T, T)'
 T add(T a, T b) { return a + b; }
   ^~~
note: constraints not satisfied
```

Named Requirements - cppreference.com

Mögliche Schreibweisen

```cpp
template <typename T>
concept arithmetic = std::is_arithmetic_v<T>;

template<arithmetic T>
T add1(T a, T b) {
  return a + b;
}

template<typename T>
requires arithmetic<T>
T add2(T a, T b) {
  return a + b;
}

template<typename T>
T add3(T a, T b) requires arithmetic<T> {
  return a + b;
}

auto add4(arithmetic auto a, arithmetic auto b) {
  // NOTE: decltype(a) != decltyp(b) possible
  return a + b;
}
```

- Verschiedene Schreibweisen für Konzepte und Anforderungen möglich

```cpp
int main() {
  SHOW(add1(1, 2));
  SHOW(add2(1, 2));
  SHOW(add3(1, 2));
  SHOW(add4(1, 2));
}
```

```
add1(1, 2) = 3
add2(1, 2) = 3
add3(1, 2) = 3
add4(1, 2) = 3
```

Beispiel: Formatter für eigenen Typ

```
struct Complex {
  double re {};
  double im {};

  Complex(double re = {}, double im = {})
    : re{re}, im{im} {}

  template<std::floating_point U>
  Complex(U re = {}, U im = {})
    requires std::convertible_to<U, double>
  : re{re}, im{im} {}
};
```

```
template <>
struct std::formatter<Complex>
  : std::formatter<std::string> {

  auto format(Complex c, auto &ctx) const {
    return std::formatter<std::string>::format(
      std::format("({}, {})", c.re, c.im), ctx);
  }
};

int main() {
  Complex c {1.1, 2.2};
  std::println("{}", c);
  std::cout << std::format("{}", c);
  // std::cout << c; // XXX: no operator <<
}
```

```
(1.1, 2.2)
(1.1, 2.2)
```

Beispiel: Schablone für komplexe Zahlen

```cpp
template <std::floating_point T = double>
struct Complex {
  T re {};
  T im {};

  Complex(T re = {}, T im = {})
    : re{re}, im{im} {}

  template<std::floating_point U>
  Complex(U re = {}, U im = {})
    requires std::convertible_to<U, T>
  : re{re}, im{im} {}
};
```

```cpp
template<typename T>
struct std::formatter<Complex<T>>
  : std::formatter<std::string> {

  auto format(Complex<T> c, auto &ctx) const {
    return std::formatter<std::string>::format(
      std::format("({}, {})", c.re, c.im), ctx);
  }
};

int main() {
  Complex c {1.1f, 2.2f};
  std::println("{}", c);
  std::cout << std::format("{}", c);
}
```

```
(1.1, 2.2)
(1.1, 2.2)
```